

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НОВОСИБИРСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»

ИНСТИТУТ ИНТЕЛЛЕКТУАЛЬНОЙ РОБОТОТЕХНИКИ

09.03.03 – Прикладная информатика

Направление (профиль): Прикладной искусственный интеллект

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

Тема работы:

TV-TENNIS

Ларионова Ирина Дмитриевна, 25943

Афанасьева Екатерина Александровна, 25943

Новосибирск
2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 HARWARE	4
1.1 clock_frequency_divider	4
1.2 player	5
1.3 ai	6
1.4 ball	7
1.5 collision check	8
1.6 dX	9
1.7 dY	10
1.8 Bresenham	11
1.9 freeze.....	12
1.10 game_active	13
1.11 player_score/game_speed	14
1.12 PROCESSOR_LOGIC	15
1.13 init_records_table	17
1.14 symbol_is_valid.....	18
1.15 name_is_valid	18
1.16 kbd_controller.....	19
1.17 MAIN_LOGIC	20
1.18 Main.....	22
2 SOFTWARE.....	24
3 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ	28
ЗАКЛЮЧЕНИЕ.....	30

ВВЕДЕНИЕ

На сегодняшний день разработка простых и понятных игр остается одним из лучших способов освоения принципов программирования. Одной из таких классических игр является разрабатываемое программное средство, которое представляет собой симулятор настольного тенниса, где основное внимание уделено плавности движений и точности отскока мяча. Особенностью проекта является реализация игровой логики, графики и логики взаимодействия с пользователем на низком уровне, что позволяет изучить принципы построения вычислительных систем и микропрограммного управления.

Целью курсового проекта является разработка полноценного аппаратно-программного комплекса, реализующего аркадную игру «TV-Tennis» для одного игрока против AI.

Для достижения цели необходимо реализовать следующие компоненты:

1. Генерация видеоизображения на LED-матрице: игровое поле, две ракетки (игрока и бота), мяч;
2. Реализация физики движения мяча с зонированием ракеток и использованием алгоритма Брезенхема для расчета углов отражения;
3. Управление движением ракетки игрока с помощью кнопок;
4. Управление движением ракетки бота, которое анализирует траекторию мяча;
5. Ввод и валидация имени пользователя через буферную клавиатуру;
6. Вывод счета и таблицы рекордов;
7. Динамическое изменение скорости игры в зависимости от счета;
8. Интеграция в схему учебного процессора CdM-8;
9. Разработка программы на ассемблере для процессора CdM-8;
10. Обработка прерывания по окончании игры;
11. Реализация системы ведения таблицы рекордов (вставка, сортировка);

Для аппаратной части разработки проекта используется эмулятор цифровой логики Logisim, а весь программный модуль реализован на языке ассемблер в среде разработки CocolIDE.

1 HARWARE

Аппаратная часть схемы построена по модульному принципу в Logisim. Основным управляющим модулем является MAIN_LOGIC, который связывает все компоненты, диаграмма связей которых представлена на рисунке 1.

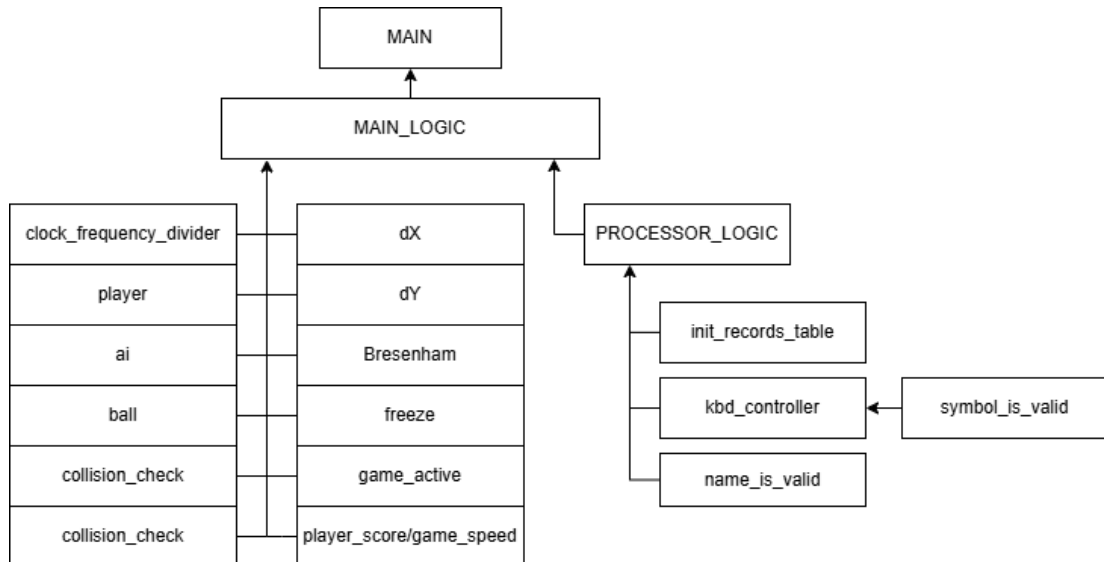


Рисунок 1 - Диаграмма компонентов

Далее в следующих подразделах представлено более подробное описание всех частей системы.

1.1 clock_frequency_divider

Подсхема «clock_frequency_divider» отвечает разделением тактовой частоты на игровую и процессорную.

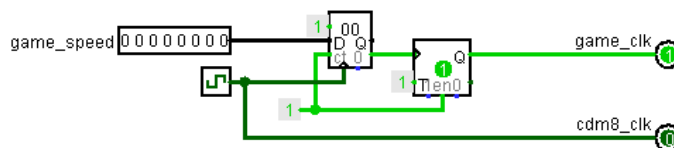


Рисунок 2 - Подсхема «clock_frequency_divider»

Входные данные подсхемы «clock_frequency_divider»:

- 8-битный контакт game_speed – константа деления тактовой частоты.

Выходные данные подсхемы «clock_frequency_divider»:

- 1-битный контакт game_clk – тактовая частота игры;
- 1-битный контакт cdm8_clk – тактовая частота процессора.

1.2 player

Подсхема «player» отвечает за управление ракеткой игрока, и вывод на игровую матрицу (см. рисунок 3). Фиксирует нажатия кнопок «влево» и «вправо», обновляет положение ракетки по оси X, позволяя игроку перемещать еще вдоль нижнего края экрана. Когда игра неактивна, регистр координаты инициализируется начальным значением.

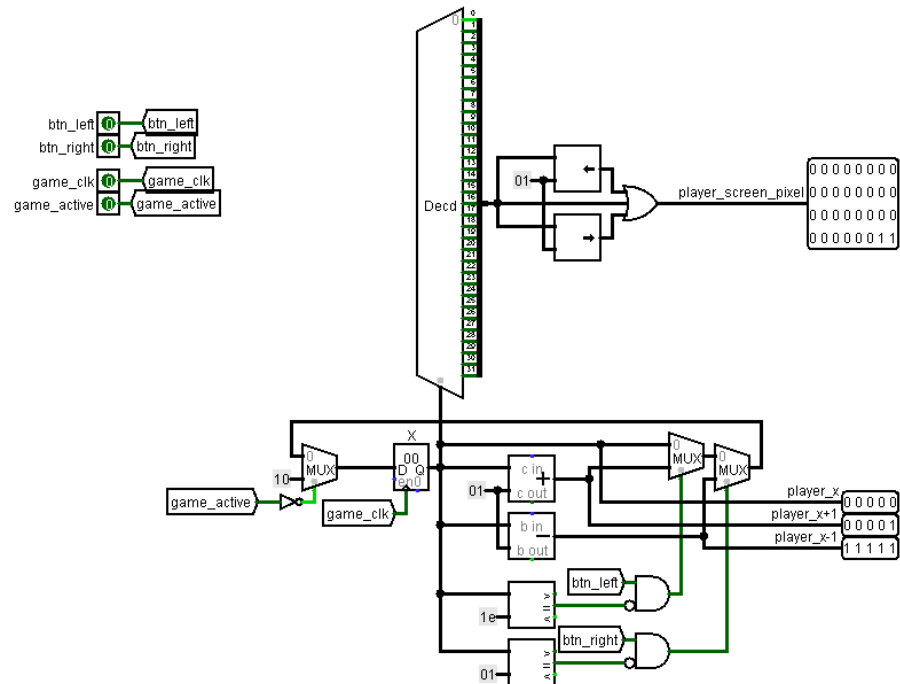


Рисунок 3 - Подсхема «player»

Входные данные подсхемы «player»:

- 1-битный контакт btn_left – сигнал кнопки влево;
- 1-битный контакт btn_right – сигнал кнопки вправо;
- 1-битный контакт game_clk – тактовая частота игры;
- 1-битный контакт game_active – флаг, активна или неактивна игра.

Выходные данные подсхемы «player»:

- 32-битный контакт player_screen_pixel – строка игрового поля с ракеткой игрока;
- 5-битный контакт player_x – координата центра ракетки игрока;
- 5-битный контакт player_x+1 – координата левого края ракетки игрока;
- 5-битный контакт player_x-1 – координата правого края ракетки игрока.

1.3 ai

Подсхема «ai» – это автоматический противник, который управляет движением верхней ракетки. В отличие от простого следования за мячом, её логика основана на аппаратном предсказании: анализируя направление и текущие координаты мяча, вычисляет точку его будущего удара о верхнюю границу (Pred_X) с учётом рикошетов от боковых стен, сравнивая с положением ракетки игрока, что позволяет ракетке заранее занимать верную позицию для перехвата. И регистр координаты инициализируется начальным значением, когда игра неактивна (см. рисунок 4).

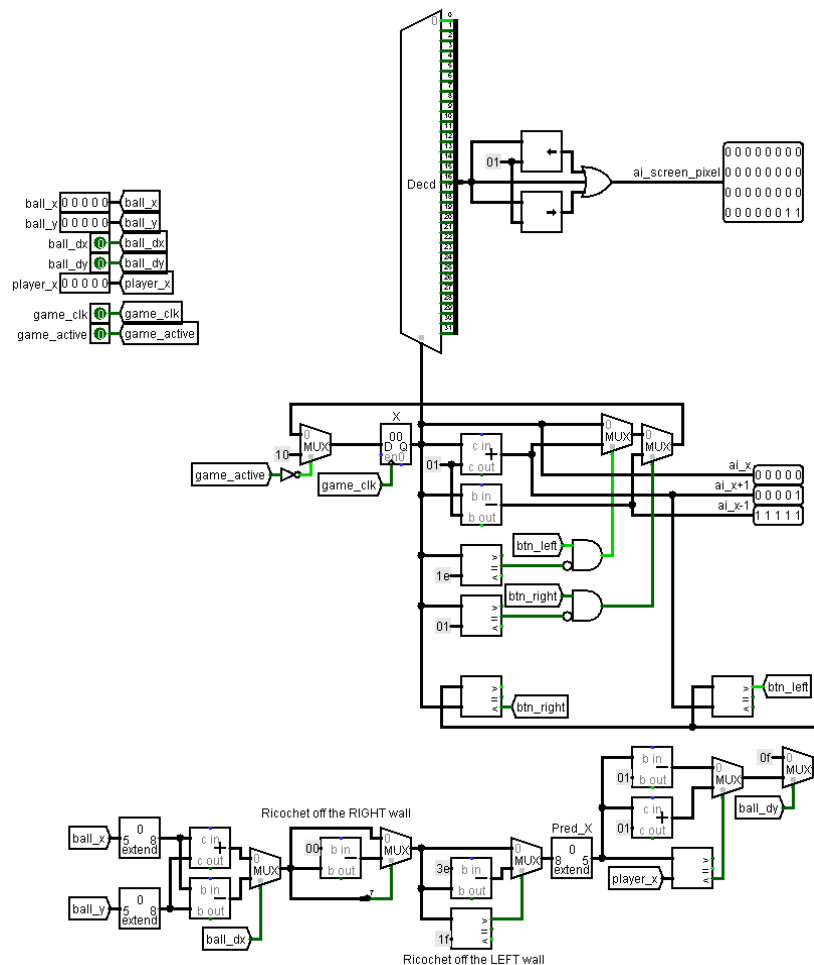


Рисунок 4 - Подсхема «ai»

Входные данные подсхемы «ai»:

- 5-битный контакт ball_x – координата мяча по оси X;
- 5-битный контакт ball_y – координата мяча по оси Y;
- 1-битный контакт ball_dx – направление мяча по оси X;
- 1-битный контакт ball_dy – направление мяча по оси Y;
- 5-битный контакт player_x – координата центра ракетки игрока по оси X;

- 1-битный контакт game_clk – тактовая частота игры;
- 1-битный контакт game_active – флаг, активна или неактивна игра.

Выходные данные подсхемы «ai»:

- 32-битный контакт ai_screen_pixel – строка игрового поля с ракеткой бота;
- 5-битный контакт ai_x – координата центра ракетки бота;
- 5-битный контакт ai_x+1 – координата левого края ракетки бота;
- 5-битный контакт ai_x-1 – координата правого края ракетки бота.

1.4 ball

Подсхема «ball» – выполняет роль контроллера мяча, преобразует 5-битные координаты в сигнал для зажигания пикселей матрицы с помощью дешифраторов и буферов. При этом аппаратно реализует алгоритм Брезенхэма, используя внутренние регистры ошибки Error_Reg_X и Error_Reg_Y, и механику «прилипания» мяча к ракетке при поступлении сигнала freeze для подкрутки. При поступлении сигнала, что игра неактивна, регистры инициализируются начальными значениями (см. рисунок 5).

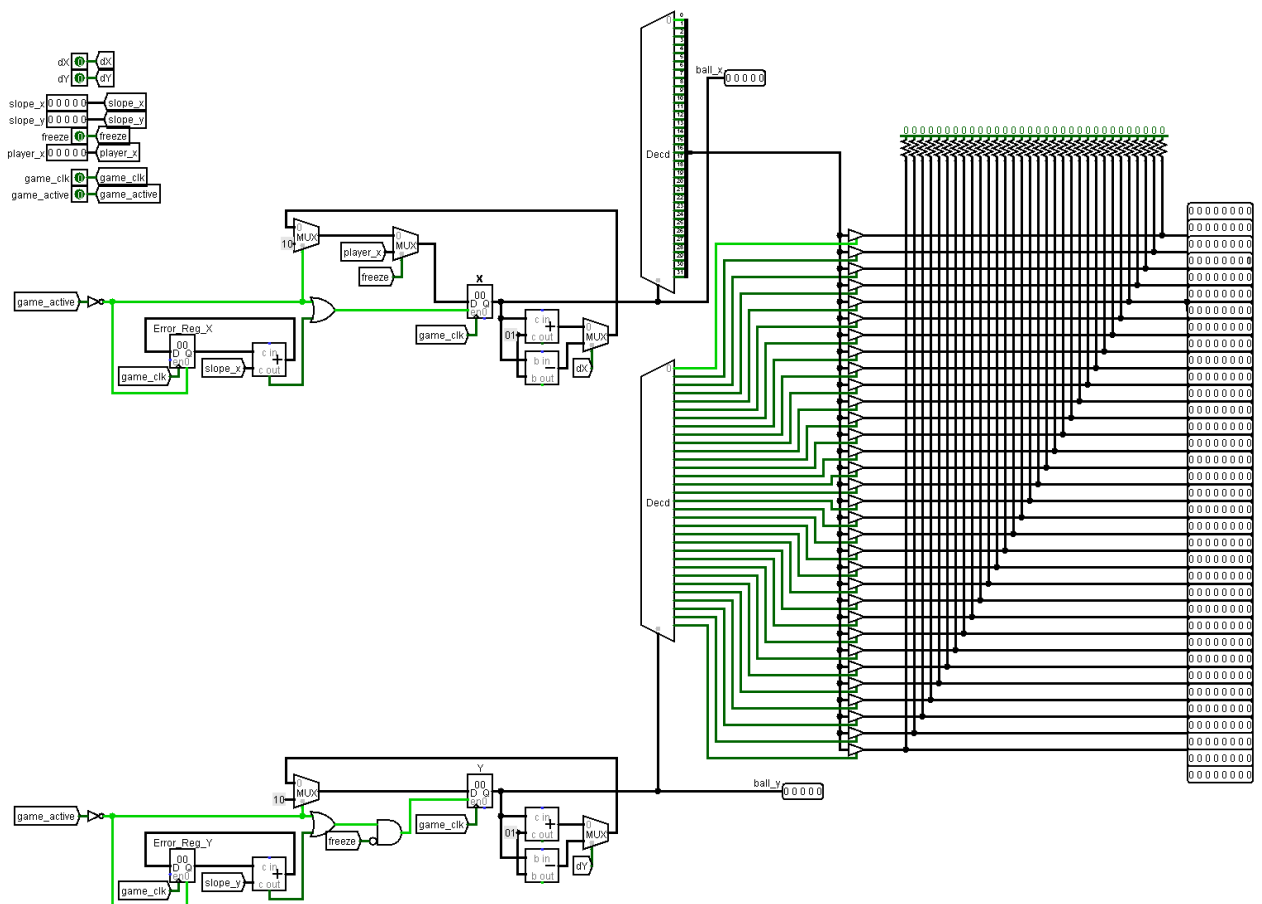


Рисунок 5 - Подсхема «ball»

Входные данные подсхемы «ball»:

- 1-битный контакт dx – направление мяча по оси X;
- 1-битный контакт dy – направление мяча по оси Y;
- 5-битный контакт slope_x – наклон траектории по оси X;
- 5-битный контакт slope_y – наклон траектории по оси Y;
- 1-битный контакт freeze – механика «прилипания» мяча к ракетке игрока;
- 5-битный контакт player_x – координата центра игрока по оси X;
- 1-битный контакт game_clk – тактовая частота игры;
- 1-битный контакт game_active – флаг, активна или неактивна игра.

Выходные данные подсхемы «ball»:

- 32 штуки 32-битных контактов – строки игрового поля с мячом;
- 5-битный контакт ball_x – координата мяча по оси X;
- 5-битный контакт ball_y – координата мяча по оси Y.

1.5 collision check

Подсхема «collision_check» – отвечает за обнаружение столкновений мяча с ракеткой игрока или бота, проверяя совпадение положения мяча с каждым из трёх сегментов ракетки по обеим осям, и формирует сигналы, что попадание произошло (см. рисунок 6).

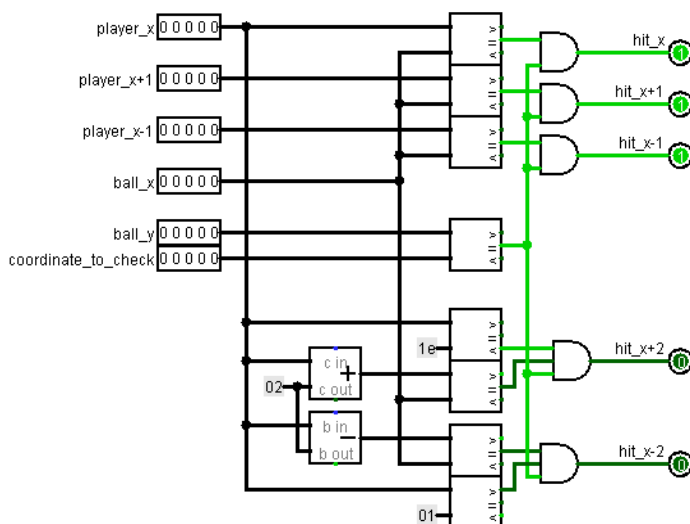


Рисунок 6 - Подсхема «collision check»

Входные данные подсхемы «collision_check»:

- 5-битный контакт `player_x` – координата центра ракетки;
- 5-битный контакт `player_x+1` – координата левого края ракетки;

- 5-битный контакт player_x-1 – координата правого края ракетки;
- 5-битный контакт ball_x – координата мяча по оси X;
- 5-битный контакт ball_y – координата мяча по оси Y;
- 5-битный контакт coordinate_to_check – координата для проверки мяча по оси Y.

Выходные данные подсистемы «collision_check»:

- 1-битный контакт hit_x – столкновение по центру ракетки;
- 1-битный контакт hit_x+1 – столкновение с левого края ракетки;
- 1-битный контакт hit_x-1 – столкновение с правого края ракетки;
- 1-битный контакт hit_x+2 – столкновение в левый угол ракетки;
- 1-битный контакт hit_x-2 – столкновение в правый угол ракетки.

1.6 dX

Подсистема «dX» управляет горизонтальным направлением движения мяча и обрабатывает все связанные с этим события: на основе сигналов о столкновениях с ракетками игрока и формирует center_hit_any, left_hit_any, right_hit_any, тем самым осуществляя зонирование ракетки, а также с левой и правой стенами EVENT_HIT_LEFT_WALL, EVENT_HIT_RIGHT_WALL. С помощью приоритетного шифратора и мультиплексора формирует итоговый сигнал dX, который указывает, должен ли мяч на следующем такте сменить направление; также схема инвертирует направление при нажатии кнопок игрока btn_left, btn_right в момент заморозки freeze, а её работа синхронизируется сигналом game_clk (см. рисунок 7).

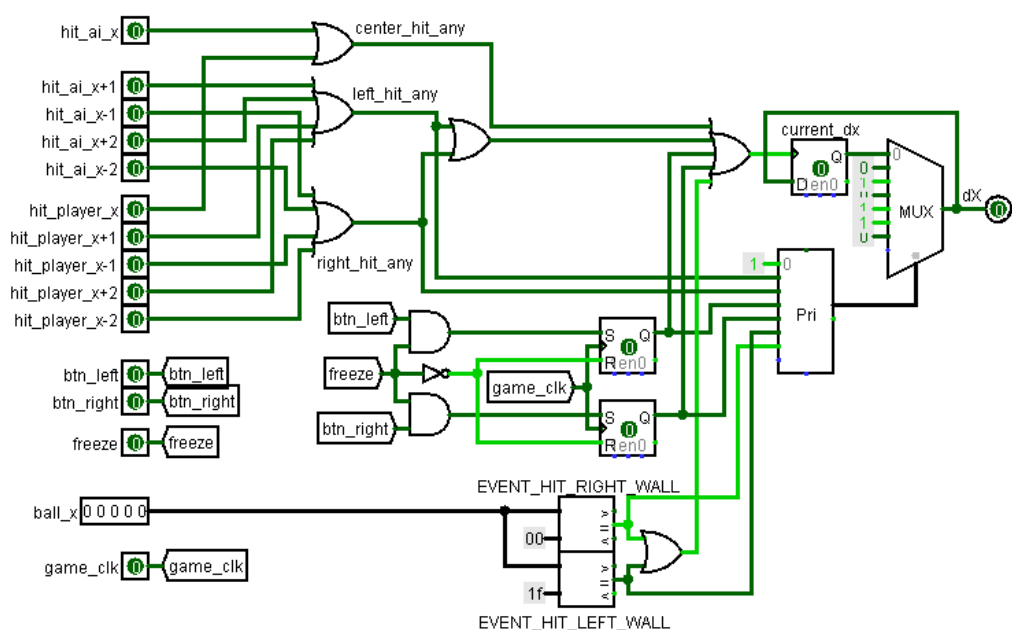


Рисунок 7 - Подсистема «dX»

Входные данные подсхемы «dX»:

- 1-битный контакт hit_ai_x – столкновение по центру ракетки бота;
- 1-битный контакт hit_ai_x+1 – столкновение с левого края ракетки бота;
- 1-битный контакт hit_ai_x-1 – столкновение с правого края ракетки бота;
- 1-битный контакт hit_ai_x+2 – столкновение в левый угол ракетки бота;
- 1-битный контакт hit_ai_x-2 – столкновение в правый угол ракетки бота;
- 1-битный контакт hit_player_x – столкновение по центру ракетки игрока;
- 1-битный контакт hit_player_x+1 – столкновение с левого края ракетки игрока;
- 1-битный контакт hit_player_x-1 – столкновение с правого края ракетки игрока;
- 1-битный контакт hit_player_x+2 – столкновение в левый угол ракетки игрока;
- 1-битный контакт hit_player_x-2 – столкновение в правый угол ракетки бота;
- 1-битный контакт btn_left – сигнал кнопки влево;
- 1-битный контакт btn_right – сигнал кнопки вправо;
- 1-битный контакт freeze – сигнал о столкновении мяча с ракеткой игрока в момент, если зажата кнопка управления;
- 5-битный контакт ball_x – координата мяча по оси X;
- 1-битный контакт game_clk – флаг, активна или неактивна игра.

Выходные данные подсхемы «dX»:

- 1-битный контакт dX – направление мяча по оси X.

1.7 dY

Подсхема «dY» управляет вертикальным направлением движения мяча: она принимает сигналы о попадании в ракетку игрока hit_player или бота hit_ai, объединяет их через логическое «ИЛИ», формируя общий сигнал hit_any, и переключает D-триггер, который меняет своё состояние при каждом таком событии, меняя тем самым направление полёта мяча по вертикали; также учитывается сигнал game_active – пока игра неактивна, смена направления заблокирована, а итоговый сигнал dY указывает, вверх или вниз мяч должен двигаться на следующем такте (см. рисунок 8).

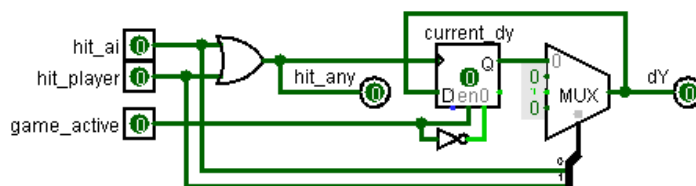


Рисунок 8 - Подсхема «dY»

Входные данные подсхемы «dY»:

- 1-битный контакт hit_ai – столкновение с ракеткой бота;
- 1-битный контакт hit_player – столкновение с ракеткой игрока;
- 1-битный контакт game_active – флаг, активна или неактивна игра.

Выходные данные подсхемы «dY»:

- 1-битный контакт dY – направление мяча по оси Y;
- 1-битный контакт hit_any – сигнал о столкновении с любой из ракеток.

1.8 Bresenham

Подсхема «Bresenham» вычисляет параметры наклона траектории мяча slope_x, slope_y, после его столкновения с ракеткой. Она принимает детализированные сигналы о том, в какую именно часть ракетки попал мяч от hit_player_x-2 до hit_player_x+2, и, аналогично для бота, разделяет их на две группы – краевые border_hit и угловые corner_hit попадания, и с помощью мультиплексоров, управляемых двухбитным регистром Angle_Reg, выбирает одно из предустановленных значений наклона (константы 0x08, 0x10, 0x1f), формируя тем самым различные углы отскока – от пологого до крутого.

Работа синхронизирована сигналами game_clk и game_active, а сам регистр угла обновляется при поступлении любого сигнала попадания, что позволяет динамически менять траекторию мяча в зависимости от точности удара (см. рисунок 9).

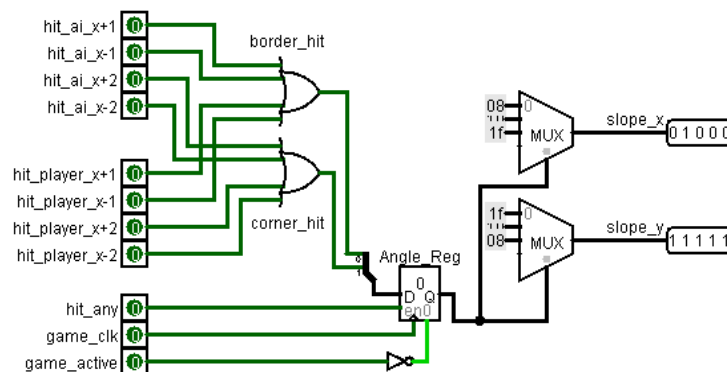


Рисунок 9 - Подсхема «Bresenham»

Входные данные подсхемы «Bresenham»:

- 1-битный контакт hit_ai_x+1 – столкновение с левого края ракетки бота;
- 1-битный контакт hit_ai_x-1 – столкновение с правого края ракетки бота;
- 1-битный контакт hit_ai_x+2 – столкновение в левый угол ракетки бота;
- 1-битный контакт hit_ai_x-2 – столкновение в правый угол ракетки бота;
- 1-битный контакт hit_player_x+1 – столкновение с левого края ракетки игрока;

- 1-битный контакт hit_player_x-1 – столкновение с правого края ракетки игрока;
- 1-битный контакт hit_player_x+2 – столкновение в левый угол ракетки игрока;
- 1-битный контакт hit_player_x-2 – столкновение в правый угол ракетки бота;
- 1-битный контакт hit_any – столкновение с любой из ракеток;
- 1-битный контакт game_clk – тактовая частота игры;
- 1-битный контакт game_active – флаг, активна или неактивна игра.

Выходные данные подсхемы «Bresenham»:

- 5-битный контакт slope_x – константа для наклона по оси X;
- 5-битный контакт slope_y – константа для наклона по оси Y.

1.9 freeze

Подсхема «freeze» реализует кратковременное залипание мяча на ракетке игрока (см. рисунок 10). Записывает сигнал о столкновении с центральной или краевой частью ракетки во время того, как зажата одна из кнопок управления, и растягивает его во времени с помощью сдвигового регистра, формируя выходной сигнал freeze, который на несколько тактов «замораживает» движение мяча по вертикали, тем самым заставляя его следовать за ракеткой (см. подсхема «Ball»).

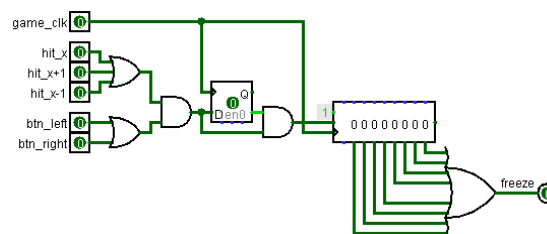


Рисунок 10 - Подсхема «freeze»

Входные данные подсхемы «freeze»:

- 1-битный контакт game_clk – тактовая частота игры;
- 1-битный контакт hit_x – столкновение по центру ракетки;
- 1-битный контакт hit_x+1 – столкновение с левого края ракетки;
- 1-битный контакт hit_x-1 – столкновение с правого края ракетки;
- 1-битный контакт btn_left – сигнал кнопки влево;
- 1-битный контакт btn_right – сигнал кнопки вправо;

Выходные данные подсхемы «dY»:

- 1-битный контакт freeze – сигнал о столкновении мяча с ракеткой игрока в момент, если зажата кнопка управления.

1.10 game_active

Подсхема «game_active» управляет состоянием игры и определяет её исход, расположенный на ней RS-триггер активирует игру, получая сигналы, что игра разрешена и введенное имя валидно, а также нажата кнопка влево или вправо. Принимая, координату мяча по вертикали ball_y, с помощью компараторов проверяет, не вылетел ли мяч за верхнюю или нижнюю границу поля. При выходе мяча за пределы схема фиксирует попадание, сбрасывая RS-триггер и увеличивая счетчики player_loser или ai_loser в зависимости от границы (см. рисунок 11).

Когда один из этих 4-битных счётчиков достигает значения 5, счётчики промахов обнуляются и активируется сигнал player_loser, сигнализируя о завершении игры и вызывая прерывание процессора для обработки счета игрока. D-триггер сигнала player_loser сбрасывается в ноль, когда процессор приступил к обновлению таблицы. Так же сигнал кнопки Reset обнуляет триггер состояния игры и счётчики.

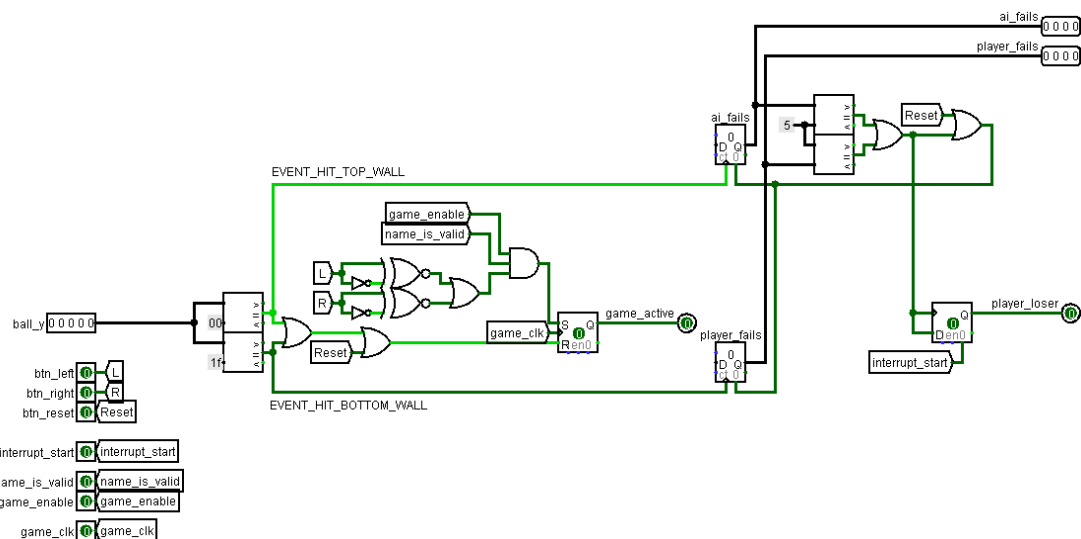


Рисунок 11 - Подсхема «game_active»

Входные данные подсхемы «game_active»:

- 5-битный контакт ball_y – координата мяча по оси Y;
- 1-битный контакт btn_left – сигнал кнопки влево;
- 1-битный контакт btn_right – сигнал кнопки вправо;
- 1-битный контакт btn_reset – сигнал кнопки сброса;
- 1-битный контакт interrupt_start – сигнал начала прерывания;
- 1-битный контакт name_is_valid – сигнал, что введенное имя валидно;
- 1-битный контакт game_enable – разрешение начала игры;
- 1-битный контакт game_clk – тактовая частота игры.

Выходные данные подсхемы «game_active»:

- 1-битный контакт game_active – флаг, активна или неактивна игра;
- 1-битный контакт player_loser – сигнал окончания игры;
- 4-битный контакт ai_fails – счёт пропущенных ботом мячей;
- 4-битный контакт player_fails – счёт пропущенных игроком мячей.

1.11 player_score/game_speed

Подсхема «player_score/game_speed» объединяет в себе подсчёт очков игрока и управление скоростью игры. При каждом успешном попадании мяча по ракетке игрока hit_player RS-триггер формирует импульс, который увеличивает два счётчика: первый count_to_5 считает до 5 последовательных отбиваний, после чего разрешает увеличение скорости, а второй – speed_change – хранит текущий уровень скорости, через мультиплексор формируя выходной сигнал game_speed из предустановленных значений (см. рисунок 12).

Параллельно с этим накапливается счёт игрока player_score, который отображается в виде двух десятичных цифр: младшие разряды 1-9 через 4-битный счётчик до 9, и старшие разряды 10-70 через второй 4-битный счётчик до 7, причём схема конкатенирует их в 8-битный формат с помощью сплиттера. Сброс счётчиков происходит по сигналам btn_reset или f1_write – вывод таблицы рекордов.

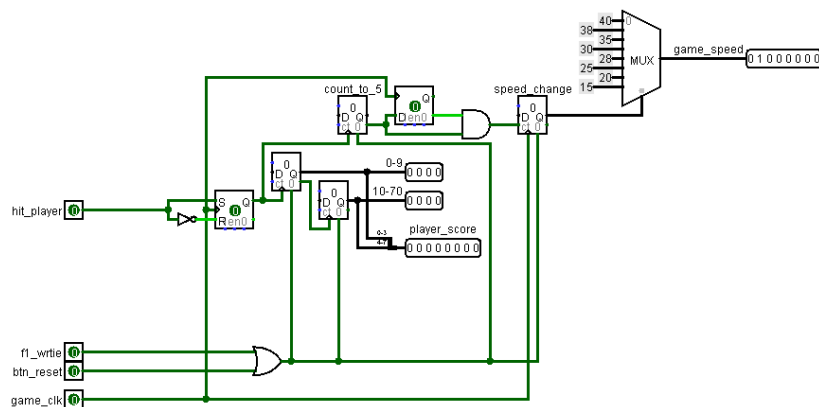


Рисунок 12 - Подсхема «player_score/game_speed»

Входные данные подсхемы «player_score/game_speed»:

- 1-битный контакт hit_player – столкновение с ракеткой игрока;
- 1-битный контакт f1_wrtie – сигнал вывода таблицы рекордов;
- 1-битный контакт btn_reset – сигнал кнопки сброса;
- 1-битный контакт game_clk – тактовая частота игры.

Выходные данные подсхемы «player_score/game_speed»:

- 4-битный контакт 0-9 – единицы очков игрока;
- 4-битный контакт 10-70 – десятки очков игрока;
- 8-битный контакт player_score – очки игрока в BCD формате;
- 8-битный контакт game_speed – константа деления тактовой частоты.

1.12 PROCESSOR_LOGIC

Подсхема «PROCESSOR_LOGIC» служит интерфейсом между процессором CdM-8 mark5 и игровой логикой (см. рисунки 13 а, б). Центральный процессор выполняет программу из встроенной ROM и использует отдельную RAM для хранения данных, включая вводимое имя и таблицу рекордов. Схема разделена на несколько функциональных блоков.

Блок ввода имени включает контроллер клавиатуры KBD_CONTROLLER и подсхему name_is_valid. Мультиплексоры и приоритетный шифратор направляют данные от клавиатуры в ОЗУ по сигналам buff_sel, buff_addr и buff_data, а по завершении ввода формируется сигнал разрешающий игру.

Блок таблицы рекордов содержит подсхему INIT_TABLE, которая при старте через сигналы init_table, init_addr и init_data заполняет ОЗУ начальными именами и счетами из своей ROM.

Арбитр прерываний реагирует на сигнал player_loser, формируя запрос прерывания IRQ и получая подтверждение IAck, после чего процессор переходит к обработчику, обновляющему таблицу рекордов; вектор прерывания задан константой `0x4`.

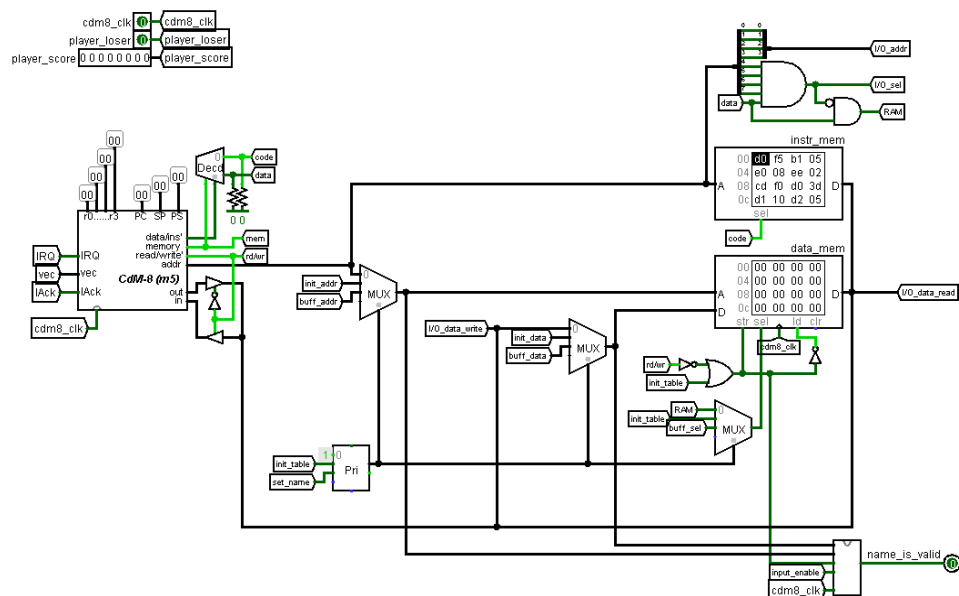


Рисунок 13 - Подсхема «PROCESSOR_LOGIC» а) CdM-8

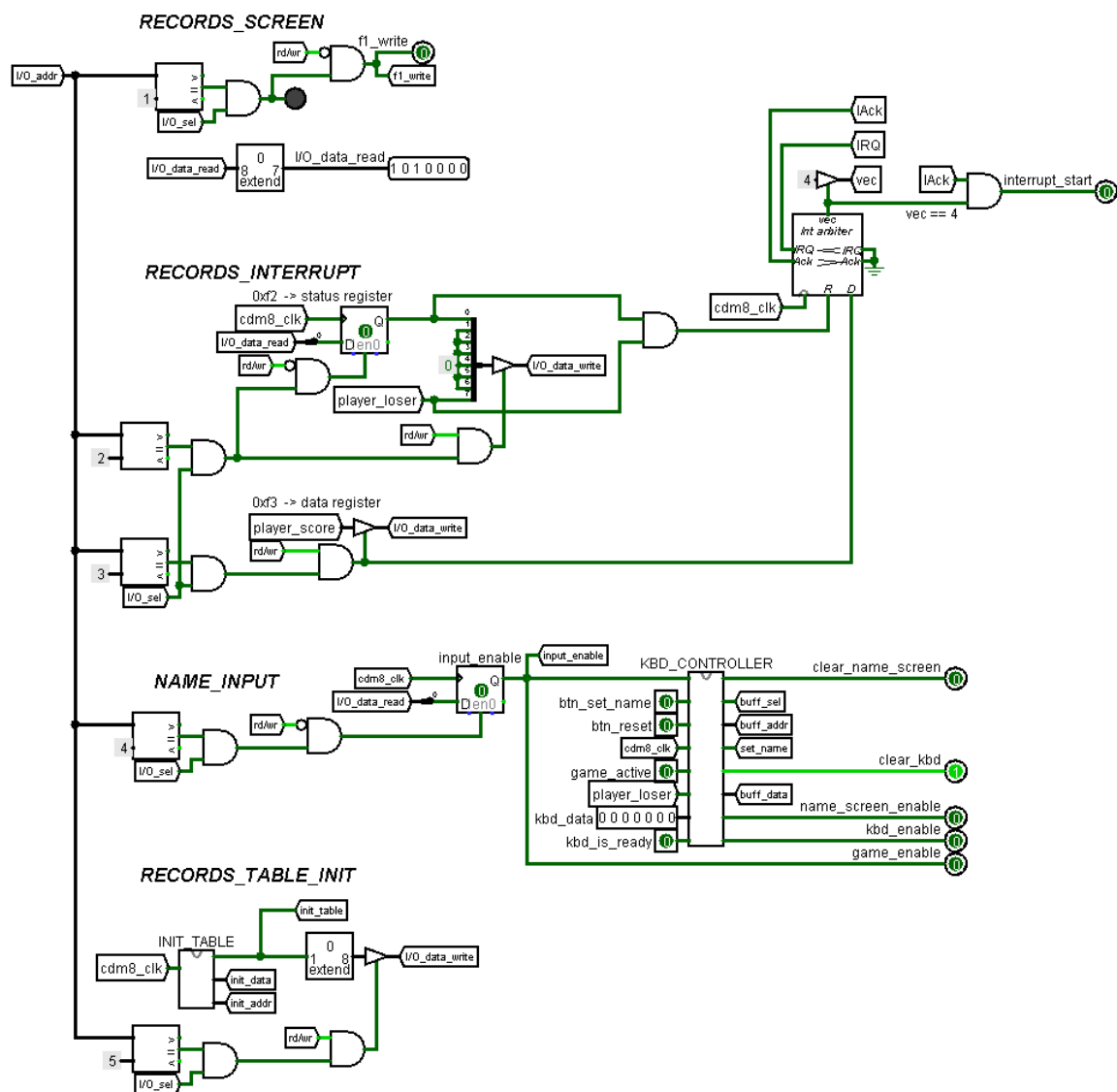


Рисунок 13 - Подсхема «PROCESSOR_LOGIC»

а) Memory-mapped I/O

Входные данные подсхемы «PROCESSOR_LOGIC»:

- 1-битный контакт `cdm8_clk` – тактовая частота процессора;
- 1-битный контакт `player_loser` – сигнал окончания игры;
- 8-битный контакт `player_score` – очки игрока;
- 1-битный контакт `btn_set_name` – сигнал кнопки ввода имени;
- 1-битный контакт `btn_reset` – сигнал кнопки сброса;
- 1-битный контакт `game_active` – флаг, активна или неактивна игра;
- 7-битный контакт `kbd_data` – данные с клавиатуры;
- 1-битный контакт `kbd_is_ready` – сигнал, что буфер клавиатуры содержит символы.

Выходные данные подсистемы «PROCESSOR_LOGIC»:

- 1-битный контакт name_is_valid – сигнал, что введенное имя валидно;
- 1-битный контакт fl_write – сигнал вывода таблицы рекордов;
- 7-битный контакт I/O_data_read – данные процессора для чтения внешними устройствами;
- 1-битный контакт interrupt_start – сигнал начала прерывания;
- 1-битный контакт clear_name_screen – сигнал очистки терминала ввода имени;
- 1-битный контакт clear_kbd – сигнал очистки буфера клавиатуры;
- 1-битный контакт name_screen_enable – сигнал, что терминал имени доступен для записи;
- 1-битный контакт kbd_enable – сигнал, что клавиатура доступна;
- 1-битный контакт game_enable – разрешение начала игры.

1.13 init_records_table

Подсхема «init_records_table» инициализирует таблицу рекордов начальными значениями при старте системы. Она построена на 6-битном счётчике, который последовательно перебирает ячейки памяти, и встроенной ROM, хранящей предопределённые имена игроков. D-триггер синхронизирует работу, а элемент «НЕ» формирует сигнал завершения, когда счётчик достигает конечного значения. Схема позволяет корректно заполнить область памяти таблицы рекордов начальными значениями до начала игры (см. рисунок 14).

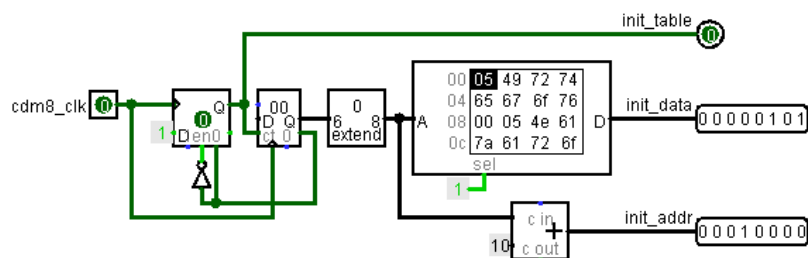


Рисунок 14 - Подсхема «init_records_table»

Входные данные подсистемы «init_records_table»:

- 1-битный контакт cdm8_clk – тактовая частота процессора.

Выходные данные подсистемы «init_records_table»:

- 1-битный контакт init_table – сигнал инициализации таблицы рекордов;
- 8-битный контакт init_data – данные для инициализации таблицы рекордов;
- 8-битный контакт init_addr – адреса инициализации таблицы рекордов.

1.14 symbol_is_valid

Подсхема «symbol_is_valid» проверяет, является ли введенный с клавиатуры символ допустимым для имени игрока. Она получает код символа и с помощью набора компараторов проверяет его принадлежность к разрешенным диапазонам: заглавные буквы A-Z, строчные буквы a-z, а также символы пробела, нижнего подчеркивания и стирания. Все результаты через элемент «ИЛИ» собираются в единый выходной сигнал is_valid, который равен единице только если символ соответствует хотя бы одному из разрешенных (см. рисунок 15).

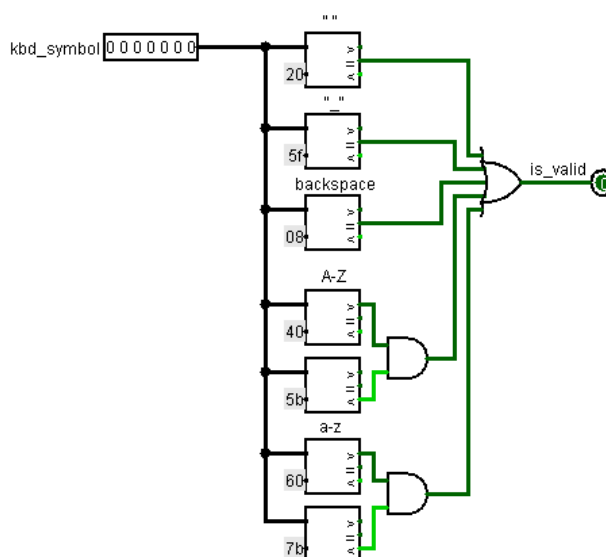


Рисунок 15 - Подсхема «symbol_is_valid»

Входные данные подсхемы «symbol_is_valid»:

- 7-битный контакт kbd_symbol – ASCII символ, введенный с клавиатуры.

Выходные данные подсхемы «symbol_is_valid»:

- 1-битный контакт is_valid – сигнал, что символ валиден.

1.15 name_is_valid

Подсхема «name_is_valid» проверяет, соответствует первый символ буфера имени правилам, что имя не должно начинаться с пробела, пустого символа или нижнего подчеркивания. Элемент «НЕ-ИЛИ» с тремя входами объединяет все возможные ошибочные комбинации, формируя итоговый сигнал `name\_is\_valid`, который равен единице, только если имя прошло все проверки успешно (см. рисунок 16).

Входные данные подсистемы «kbd_controller»:

- 1-битный контакт input_enable – сигнал разрешения ввода имени;
- 1-битный контакт btn_set_name – сигнал кнопки ввода имени;
- 1-битный контакт btn_reset – сигнал кнопки сброса;
- 1-битный контакт cdm8_clk – тактовая частота процессора;
- 1-битный контакт game_active – флаг, активна или неактивна игра;
- 1-битный контакт player_loser – сигнал окончания игры;
- 7-битный контакт kbd_data – данные с клавиатуры;
- 1-битный контакт kbd_is_ready – сигнал, что буфер клавиатуры содержит символы.

Выходные данные подсистемы «kbd_controller»:

- 1-битный контакт set_name – сигнал ввода имени;
- 1-битный контакт name_screen_enable – сигнал, что терминал имени доступен для записи;
- 1-битный контакт kbd_enable – сигнал, что клавиатура доступна;
- 1-битный контакт clear_name_screen – сигнал очистки терминала ввода имени;
- 1-битный контакт clear_kbd – сигнал очистки буфера клавиатуры;
- 1-битный контакт buff_sel – сигнал записи в буфер имени;
- 8-битный контакт buff_addr – адрес буфера имени;
- 8-битный контакт buff_data – данные для записи в буфер имени.

1.17 MAIN_LOGIC

Подсистема MAIN_LOGIC — это центральный модуль, который объединяет все компоненты игры в единую систему (см. рисунок 18), внутри происходит маршрутизация управляющих сигналов, тактовых импульсов и данных между игровыми модулями модулями, а также с процессорной частью PROCESSOR_LOGIC.

Благодаря этому достигается целостность игрового процесса: координаты мяча и ракеток передаются в модули коллизий, сигналы попаданий обрабатываются для смены направления и расчёта углов отскока, а сформированные 32-битные строки пикселей от каждого игрового объекта накладываются друг на друга и выводятся на LED-матрицу.

Через подсистему передаются сигналы прерывания player_loser, текущий счёт игрока player_score, данные с клавиатуры для ввода имени, а также сигналы разрешения игры game_enable, валидации имени name_is_valid и вывода таблицы рекордов fl_write.

Таким образом, MAIN_LOGIC не только координирует движение мяча и обработку столкновений в реальном времени, но и обеспечивает корректное взаимодействие с

подсистемой ввода имени, хранения рекордов и обработки прерываний, являясь связующим звеном между «железом» и программным кодом.

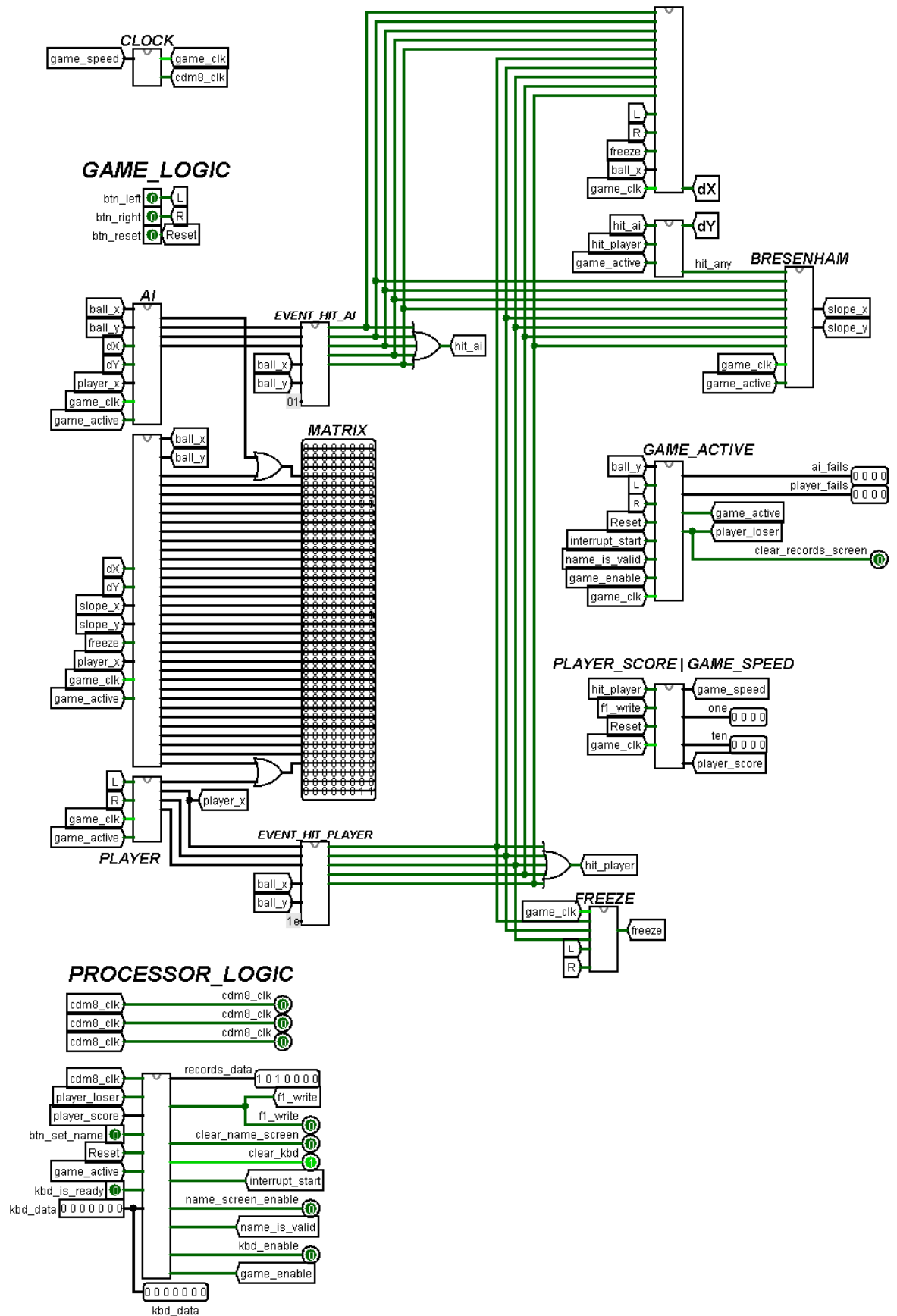


Рисунок 18 - Подсхема «MAIN_LOGIC»

Входные данные подсистемы «MAIN_LOGIC»:

- 1-битный контакт btn_left – сигнал кнопки влево;
- 1-битный контакт btn_right – сигнал кнопки вправо;
- 1-битный контакт btn_reset – сигнал кнопки сброса;
- 1-битный контакт btn_set_name – сигнал кнопки ввода имени;
- 1-битный kbd_is_ready – сигнал, что буфер клавиатуры содержит символы;
- 7-битный контакт kbd_data – данные с клавиатуры.

Выходные данные подсистемы «MAIN_LOGIC»:

- 32 штуки 32-битных контактов – строки игрового поля с мячом;
- 4-битный контакт ai_fails – счёт пропущенных ботом мячей;
- 4-битный контакт player_fails – счёт пропущенных игроком мячей;
- 1-битный контакт clear_records_screen – сигнал очистки терминала таблицы рекордов;
- 4-битный контакт one – единицы очков игрока;
- 4-битный контакт ten – десятки очков игрока;
- 3 штуки 1-битных контактов cdm8_clk – тактовая частота процессора;
- 7-битный контакт records_data – данные таблицы рекордов;
- 1-битный контакт f1_write – сигнал вывода таблицы рекордов;
- 1-битный контакт clear_name_screen – сигнал очистки терминала ввода имени;
- 1-битный контакт clear_kbd – сигнал очистки буфера клавиатуры;
- 1-битный контакт name_screen_enable – сигнал, что терминал имени доступен для записи;
- 1-битный контакт kbd_enable – сигнал, что клавиатура доступна;
- 7-битный контакт kbd_data – данные с клавиатуры.

1.18 Main

Главная схема «MAIN» представляет собой верхний уровень всей игры, объединяя пользовательский интерфейс и основную логику. На ней расположены элементы ввода-вывода: LED-матрица 32x32 для игрового поля, кнопки управления L и R для перемещения ракетки игрока, кнопка Reset для сброса, а также кнопка Set_Name для ввода имени. Клавиатура используется для ввода имени игрока. Шестнадцатеричные дисплеи отображают текущий счёт игрока и бота (см. рисунок 19).

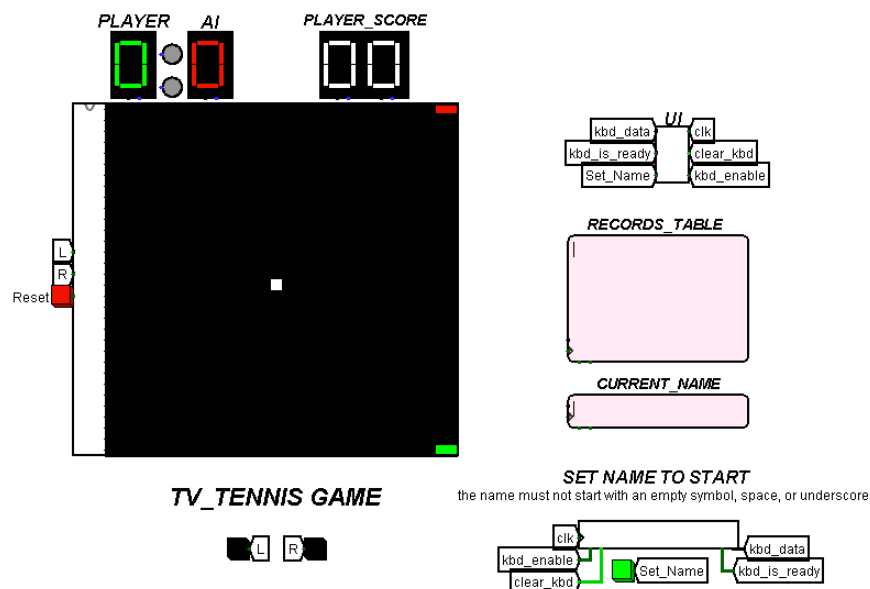


Рисунок 19 - Подсхема «MAIN»

Все эти компоненты подключены к центральному блоку MAIN_LOGIC, который реализует как аппаратную логику игры, так и взаимодействие с процессорной частью. Туннели связывают кнопки, клавиатуру и дисплеи с внутренними сигналами MAIN_LOGIC, а текстовые поля на схеме служат визуальными подсказками для пользователя.

2 SOFTWARE

Программный модуль реализует подсистему хранения, обновления и вывода таблицы рекордов, код написан на ассемблере и спроектирован с учетом жестких аппаратных ограничений.

Главной архитектурной особенностью является косвенная адресация при сортировке, то есть для изменения позиций рекордов в таблице физически перемещаются не сами данные (блоки по 9 байт), а только их адреса в массиве указателей (по 1 байту), что существенно экономит такты процессора при обновлении таблицы.

Карта памяти представлена в таблице 1, учитывая, что система активно взаимодействует с внешним оборудованием через фиксированные адреса портов:

Таблица 1 - Карта памяти процессора

	Адрес	Назначение
ROM	0x00	Начало исполняемого кода
	0xF8	Вектор прерывания, указывает на counter_handler
RAM	0x00	Буфер ввода имени name_buffer (8 байт)
	0x10	Физические данные рекордов records_data (45 байт)
	0x3D	Массив указателей на рекорды records_ptrс (5 байт, смещение 0x10 + 45)
	0xF0	Вершина стека
Memory-mapped I/O	0xF1	Терминал для печати таблицы рекордов
	0xF2	Статус-регистр для разрешения/запрета прерывания
	0xF3	Порт чтения нового счета игрока
	0xF4	Разрешение/запрет аппаратного ввода имени
	0xF5	Статус аппаратной загрузки инициализации таблицы начальными значениями

Система хранит топ-5 рекордов, где каждый рекорд состоит из 9 байт:

- 1 байт: Счет в формате BCD – двоично-десятичный код.
- 8 байт: Имя игрока – ASCII-символы.

Для управления данными используются две структуры:

- records_data (45 байт): Непрерывный блок памяти, где физически лежат данные слотов.
- records_ptrс (5 байт): Массив адресов. Каждый байт массива содержит адрес начала 9-байтового блока в records_data. Сортировка таблицы осуществляется исключительно изменением порядка адресов в этом массиве.

Жизненный цикл программы и взаимодействие с аппаратной частью:

1. Инициализация и синхронизация «start»:

- Busy-Wait цикл: Процессор опрашивает порт 0xF5, ожидая завершения работы аппаратного загрузчика, пока не будет получен 0.
 - Подготовка памяти: Стек устанавливается на 0xF0.
 - Инициализация указателей: Программа проходит по массиву records_ptrs и последовательно записывает в него адреса блоков из records_data с шагом 9 байт.
 - Запуск: Включаются внешние устройства, порты 0xF2, 0xF4, разрешаются прерывания, и процессор переходит в бесконечный режим ожидания прерываний.
2. Обработчик прерывания «counter_handler» (см. рисунок 20):
- Прерывание срабатывает, когда аппаратура сигнализирует о завершении игры и готовности нового счета.
 - Сохраняется контекст процессора и временно блокируется аппаратный ввод имени, запись 0x00 в 0xF4.
 - Счет считывается из порта 0xF3.
 - В цикле происходит сравнение нового счета с текущими результатами в таблице через разыменовывание массива records_ptrs, если новый счет строго больше одного из существующих, вызывается подпрограмма вставки insert, после чего цикл прерывается.
 - Обновленная таблица выводится на терминал 0xF1.
 - Восстанавливается контекст и разрешается ввод имени, запись 0x01 в 0xF4.

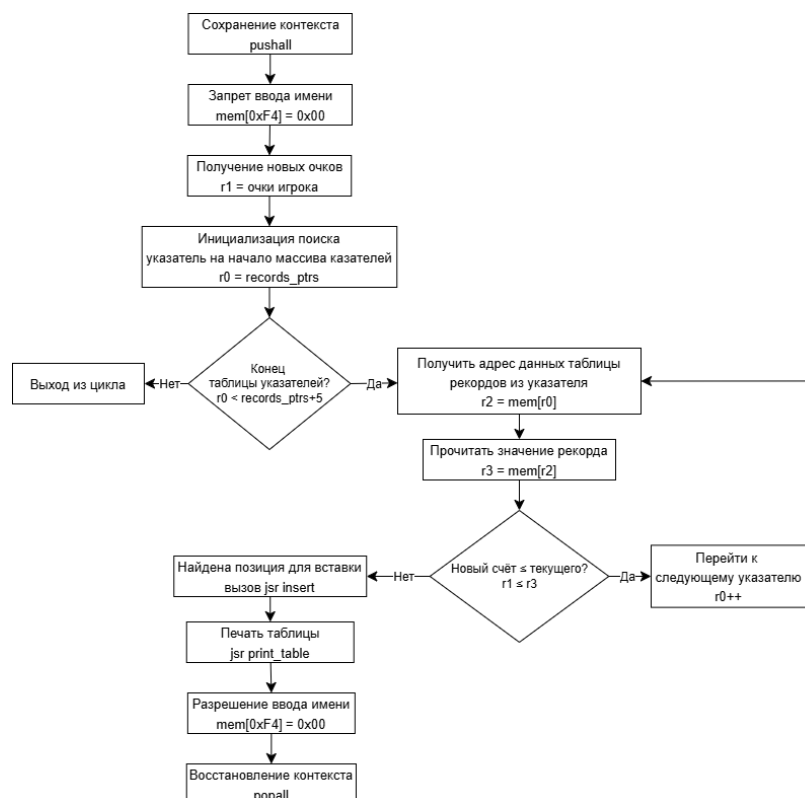


Рисунок 20 - Диаграмма обработчика прерывания «counter_handler»

3. Алгоритм вставки рекорда «insert» – реализует логику "вытеснения" слабого рекорда без копирования тяжелых блоков (см. рисунок 21):

- Извлекается указатель с последнего места таблицы – выбывающий рекорд.
- По этому адресу записывается новый счет, 1 байт.
- Туда же копируются 8 байт имени из буфера name_buffer.
- Происходит сдвиг указателей в массиве records_ptrs на одну позицию вниз, чтобы освободить место для нового рекорда.
- Обновленный указатель, бывший последним, записывается на освободившуюся позицию в records_ptrs.

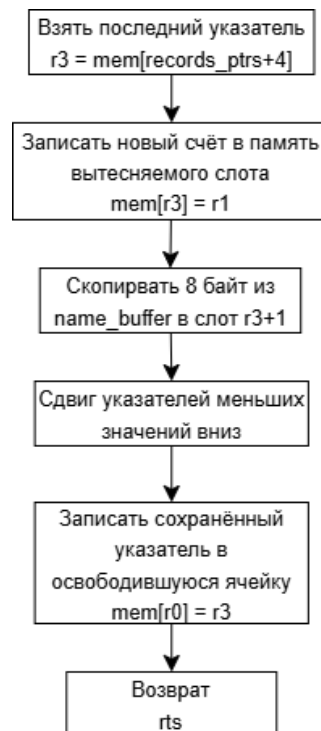


Рисунок 21 - Диаграмма алгоритма вставки рекорда «insert»

4. Подсистемы вывода:

- «print_table»: Итерируется по массиву records_ptrs. Для каждого элемента читает базовый адрес, выводит счет, печатает пробел, выводит 8 символов имени в 0xF1 и переход на новую строку.
- «print_number»: Декодер BCD-чисел. Поскольку счет хранится в виде упакованного двоично-десятичного числа, функция с помощью битовых сдвигов и масок разделяет число на десятки и единицы, прибавляя к каждой цифре смещение 0x30 для получения ASCII-кода и отправляет в терминал 0xF1.

Для более наглядного представления связи частей системы на рисунке 22 представлена диаграмма последовательности взаимодействия аппаратной и программной частей во время инициализации и обработки прерывания.

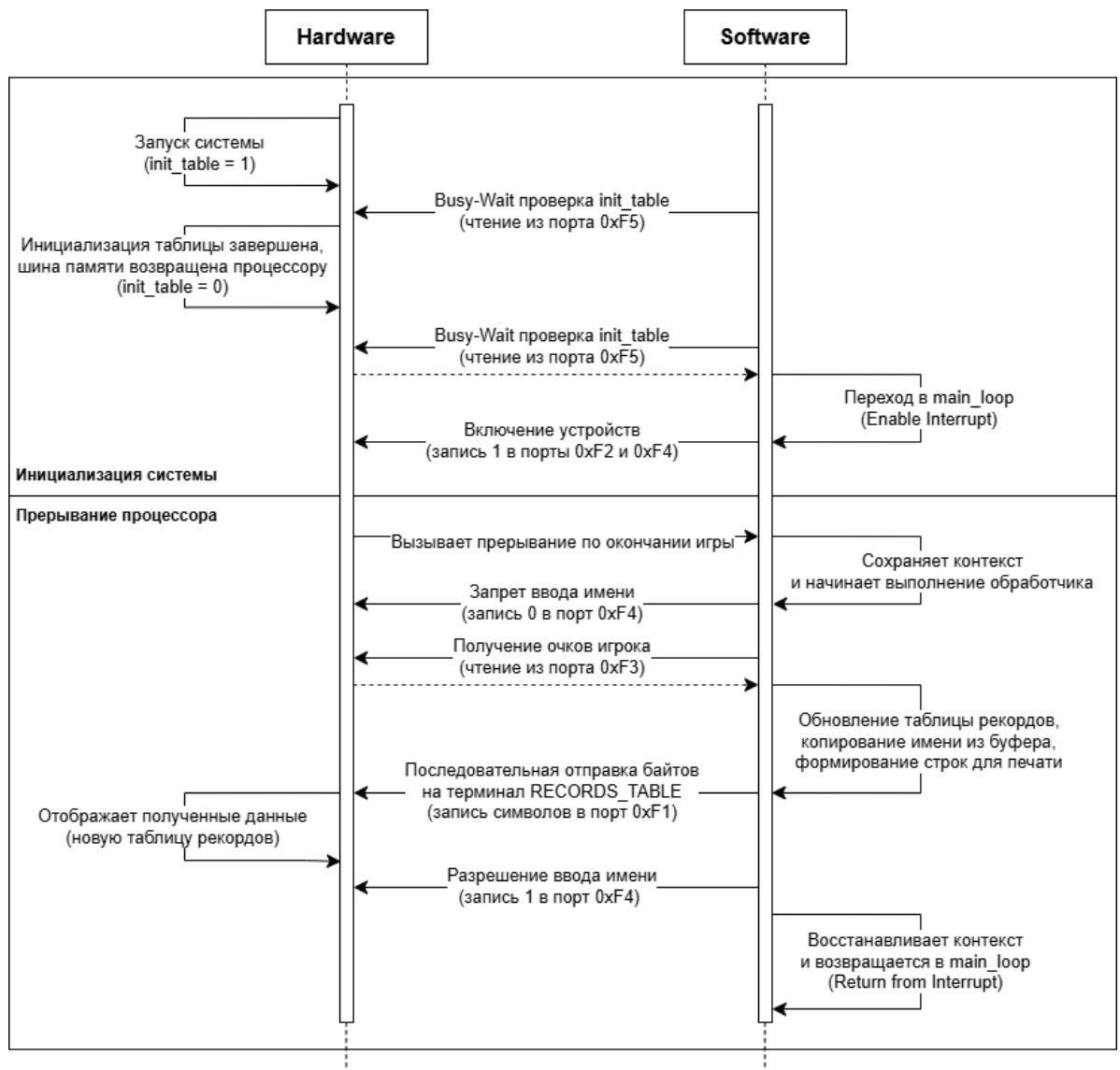


Рисунок 22 - Диаграмма последовательности взаимодействия Hardware и Software

3 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

При запуске программного средства пользователь попадает на главную схему, на которой расположены: главная игровая матрица, индикаторы очков, терминалы для ввода имени и таблицы рекордов, клавиатура, а также кнопки ввода имени, управления и сброса (см. рисунок 23).

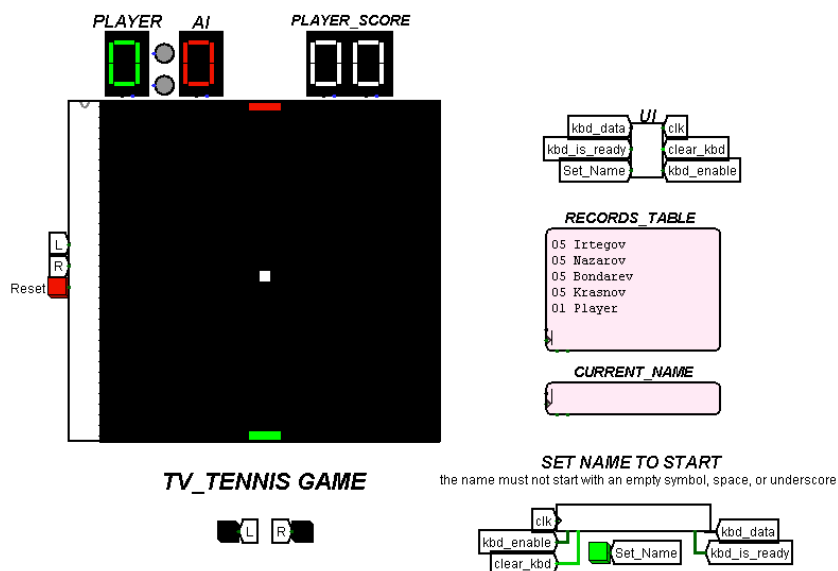


Рисунок 23 - Главная схема игры

При нажатии пользователем на кнопку «Set_Name», клавиатура становится доступна для ввода имени, которое будет записано мгновенно и отображаться в терминале CURRENT_NAME (см. рисунок 24).

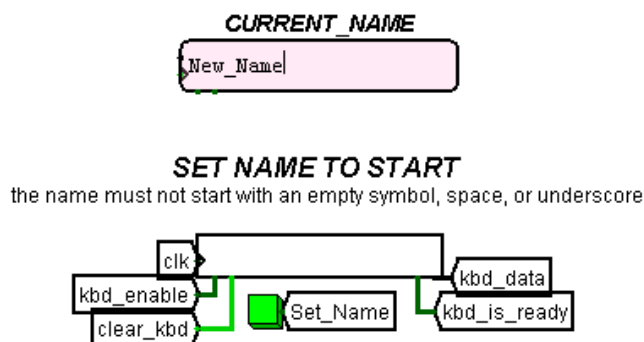


Рисунок 24 - Ввод имени

Только после ввода корректного имени, можно начать игру, нажав на кнопки «L»-влево или «R»-вправо (см. рисунок 25).



Рисунок 25 - Кнопки управления ракеткой

Вверху схемы размещено 4 шестнадцатеричных индикатора (см. рисунок 26), которые отображают количество забитых игроком и ботом мячей – «PLAYER» и «AI», а также отраженные нижней ракеткой подачи – «PLAYER_SCORE» – максимум 79.

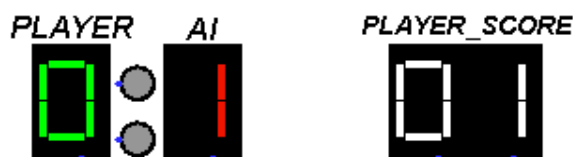


Рисунок 26 - Индикаторы счёта и очков

После того, как игрок или бот забили 5 мячей, игра останавливается и выводится обновленная таблица рекордов на терминале «RECORDS_TABLE», с новым игроком, если он набрал необходимое количество очков (см. рисунок 27).

RECORDS_TABLE	
05	Irtegov
05	Nazarov
05	Bondarev
05	Krasnov
02	New_Name

Рисунок 27 - Таблица рекордов

После вывода таблицы, счёт сбрасывается и можно продолжать играть под тем же именем либо ввести новое. Также для сброса игры и имени можно нажать кнопку «Reset», которая сбрасывает состояние системы, но не таблицы рекордов (см. рисунок 28).



Рисунок 28 - Кнопка сброса

ЗАКЛЮЧЕНИЕ

Изучение принципов работы вычислительных систем требует понимания не только программной логики, но и её тесного взаимодействия с аппаратной частью. Разработка классических игр является эффективным способом освоения фундаментальных основ программирования, так как за внешней простотой механики скрываются сложные процессы обработки физики, коллизий и управления ресурсами в реальном времени.

Целью курсовой работы стало создание игровой системы «TV-Tennis», объединяющей в себе низкоуровневое программное обеспечение и эмулируемую аппаратную логику. Разработанное средство успешно решает задачи визуализации игрового процесса, обеспечивая плавность движений и корректность физических взаимодействий, что позволяет пользователю получить интуитивно понятный игровой опыт.

В ходе выполнения работы были реализованы следующие этапы:

- Подробно изучена механика настольного тенниса, правила игры и алгоритмы движения объектов;
- Определены и заложены ключевые функциональные возможности программного модуля;
- Реализована математическая модель отскока мяча от поверхностей с учетом различных углов наклона;
- Спроектирована и внедрена удобная система управления ракеткой игрока;
- Разработан алгоритм искусственного интеллекта для ракетки противника, обеспечивающий соревновательный элемент;
- Создан пользовательский интерфейс, включающий систему ввода имени, подсчет очков и таблицу рекордов;
- Проведено комплексное тестирование системы, в ходе которого были устранены ошибки позиционирования и обработки столкновений;
- Произведена оптимизация кода для обеспечения стабильной работы в среде эмуляции.

В результате с использованием языка ассемблер в среде CosoIDE и аппаратного моделирования в Logisim была создана единая система, полностью удовлетворяющая техническим требованиям. Разработанный проект позволил на практике изучить принципы связи «железа» и программного кода. Созданная система обработки рекордов и ввода данных демонстрирует возможность реализации сложных структур данных даже в условиях ограниченных ресурсов.

Следует отметить, что игра «TV-Tennis» обладает потенциалом для дальнейшей модернизации. В перспективе функционал может быть расширен за счет добавления режима игры для двух человек или усложнения уровней сложности ИИ. Такие доработки позволят превратить учебный проект в полноценный завершённый продукт.

Реализованное программное средство представляет собой законченный симулятор, который может быть использован как база для изучения низкоуровневого программирования и проектирования цифровых устройств, демонстрируя надёжность и стабильность работы в интерактивном режиме.